Curs: Dezvoltarea Aplicațiilor Software Securizate
Facultatea de Matematică și Informatică
Universitatea din București

Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro

Proiect #2

Break the Login – Atacarea și securizarea autentificării

Obiectiv	Scopul proiectului este ca studentul să înțeleagă cum este atacat în practică un sistem de autentificare și cum trebuie implementat corect pentru a rezista atacurilor reale. Studentul va: • construi un mecanism de autentificare funcțional, dar inițial vulnerabil; • demonstra exploatarea lui prin tehnici de ethical hacking; • remedia vulnerabilitățile prin implementarea controalelor corecte de securitate; • valida că atacurile nu mai funcționează după fix. Accentul cade pe: • logică de autentificare; • parole; • sesiuni / token-uri; • resetare parolă; nu pe funcționalități de business complexe.
Scenariu	Compania fictivă "AuthX" dezvoltă o aplicație internă folosită de angajați pentru acces la resurse sensibile. Aplicația este deja folosită în producție, dar echipa de securitate a semnalat probleme serioase legate de autentificare. Studentul joacă dublu rol în proiect: • Security tester – identifică și demonstrează vulnerabilitățile; • Developer – repară implementarea și întărește sistemul. Auditul se concentrează exclusiv pe: • login; • gestionarea parolelor; • sesiuni; • resetarea parolei.

Curs: Dezvoltarea Aplicațiilor Software Securizate
Facultatea de Matematică și Informatică
Universitatea din București

Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro

Funcționalități obligatorii – Mecanism de autentificare

Nr.	Funcționalitate	Ce trebuie să implementeze studentul	Observații importante
3.1	Înregistrare utilizator (Register)	Formular de creare cont cu email/username și parolă	Input-ul trebuie validat în backend
		Stocarea utilizatorului într-o bază de date reală	DB reală (ex: PostgreSQL, SQLite)
		Asocierea unui rol simplu (ex: USER)	Rolul va fi folosit ulterior la autorizare
3.2	Autentificare (Login)	Formular de login cu username + parolă	Răspuns inițial diferențiat (vulnerabil)
		Verificarea credențialelor în backend	Fără logică doar în UI
		Crearea unei sesiuni (cookie) sau token (JWT)	Model ales clar și documentat
		Transmiterea sesiunii/token-ului către client	Cu setări inițial incomplete (intenționat)
3.3	Logout	Invalidarea sesiunii sau a token-ului	După logout sesiunea nu mai este validă
		Ștergerea cookie-ului sau marcarea token-ului ca invalid	Necesită verificare practică
3.4	Resetare parolă	Funcționalitate „Forgot password”	Implementare inițial simplificată
		Generarea unui token de resetare	Inițial predictibil sau reutilizabil
		Endpoint pentru setarea unei parole noi	Fără controale avansate la început
3.5	Gestionare sesiuni	Menținerea autentificării între request-uri	Cookie sau token trimis automat
		Asocierea request-urilor cu utilizatorul logat	Baza pentru access control

Cerințe de securitate

Nr.	Categoria	Vulnerabilitate inițială (v1)	Atac demonstrat (PoC)	Fix obligatoriu (v2)
4.1	Password Policy slab	– Sunt acceptate parole foarte scurte sau triviale– Nu există validare la înregistrare	– Creare conturi cu parole extrem de slabe– Explicarea riscului de credential stuffing / brute force	– Lungime minimă parolă– Complexitate minimă– Mesaj de eroare generic
4.2	Stocare nesigură a parolelor	– Parole stocate în clar– Sau hash slab (MD5/SHA1)	– Acces la DB (simulat) și vizualizare parole–	– Hash modern (bcrypt / argon2 / scrypt)– Salt

Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro

			Demonstrarea cât de ușor pot fi compromise	implicit– Nicio parolă în clar în DB
4.3	Brute force / lipsă rate limiting	– Număr nelimitat de încercări login– Fără blocare cont	– Script/tool simplu care încearcă parole multiple	– Rate limiting pe login– Blocare temporară după N încercări– Logarea tentativelor
4.4	User Enumeration	– Mesaje diferite pentru „user inexistent” vs „parolă greșită”	– Enumerarea utilizatorilor existenți pe baza mesajelor	– Mesaj unic: „Invalid credentials”– Timp de răspuns uniform
4.5	Gestionare nesigură a sesiunilor	– Cookie fără HttpOnly / Secure / SameSite– Token cu expirare prea lungă	– Reutilizarea sesiunii– Explicarea riscului în prezența XSS	– Cookie-uri securizate– Expirare sesiune– Rotație token la login– Invalidare la logout
4.6	Resetare parolă nesigură	– Token predictibil– Token reutilizabil– Fără expirare	– Resetarea parolei fără control corect	– Token random, one-time– Expirare scurtă– Invalidare după utilizare

Cerințe de livrare

Nr.	Livrabil	Ce trebuie să conțină	Clarificări / Criterii
1	Cod sursă	Aplicația completă funcțională	Codul trebuie să poată fi rulat local
		Două versiuni distincte	vulnerable și fixed (branch-uri sau tag-uri clare)
		Structură clară a proiectului	Fără cod duplicat inutil
2	Raport de securitate (mini pentest)	Descrierea fiecărei vulnerabilități	Mapată pe cerințele proiectului
		Pașii de exploatare (PoC)	Explicați clar, reproductibil
		Impactul vulnerabilității	Ce poate obține un atacator
		Fix-ul implementat	Explicat concis, cu referință la cod
		Re-test	Dovadă că atacul nu mai funcționează
3	Dovezi practice	Request / response relevante	Curl, Postman sau Burp
		Capturi de ecran	Burp Suite / Postman
		Exemple de input malițios	Input folosit în PoC (safe, de laborator)

Curs: Dezvoltarea Aplicațiilor Software Securizate
Facultatea de Matematică și Informatică
Universitatea din București

Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro

Ghid de implementare

Nr.	Etapa	Ce face utilizatorul	Ce face aplicația (pe scurt)	De ce contează din punct de vedere al securității
1	Acces aplicație	Deschide aplicația	Afișează pagina Register / Login	Tot input-ul din browser este nesigur
2	Register – trimitere date	Completează email + parolă	Primește datele de la client	Aici pot apărea parole slabe și input malițios
3	Register – validare	–	Validează formatul și lungimea datelor	Previne date invalide sau periculoase
4	Register – existență user	–	Verifică dacă userul există deja	Evită conflicte și scurgeri de informații
5	Register – parolă	–	Hash-uiește parola (nu o stochează în clar)	Protejează parolele dacă DB este compromis
6	Register – persistare	–	Salvează utilizatorul în baza de date	DB reală → securitate realistă
7	Login – trimitere date	Introduce user + parolă	Primește credențialele	Punct clasic de atac
8	Login – verificare user	–	Caută utilizatorul în DB	Evită user enumeration
9	Login – verificare parolă	–	Compară parola cu hash-ul	Autentificare corectă
10	Login – succes	–	Creează sesiune / token	Leagă browserul de utilizator
11	Login – răspuns	Primește confirmare	Trimite cookie / token	Setări greșite = risc major
12	După login (general)	Face orice acțiune	Verifică sesiunea/token-ul	Fără asta, oricine poate face orice
13	Identitate	–	Identifică utilizatorul curent	Bază pentru autorizare
14	Autorizare	–	Decide dacă userul are voie	Previne acces neautorizat
15	Gestionare sesiune	Navighează în aplicație	Verifică validitatea sesiunii	Previne reutilizarea
16	Logout	Apasă Logout	Invalidează sesiunea/token-ul	Previne hijacking
17	Forgot password	Cere resetare parolă	Generează token aleator	Token slab = compromitere cont
18	Reset password	Accesează link	Permite setarea noii parole	Control strict necesar
19	Final reset	–	Invalidează token-ul	Previne reutilizarea
20	Ethical hacking	Încearcă atacuri	Permite testare controlată	Demonstrează vulnerabilitățile
21	Fix	–	Adaugă protecții (rate limit, mesaje generice etc.)	Întărește sistemul

Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro

22	Re-test	Repetă atacurile	Verifică că nu mai funcționează	Confirmă securizarea
23	Mentalitate	–	Gândește ca un atacator	Secure by design

Structură minimă de bază de date (nivel mediu, realist)

1. Tabelul users

Câmp	Tip (exemplu)	Descriere
id	int / uuid (PK)	Identificator unic utilizator
email	string (unique)	Username / email
password_hash	string	Hash parolă (bcrypt / argon2)
role	enum	ANALYST / MANAGER
created_at	timestamp	Data creării contului
locked	boolean	Cont blocat (brute-force)

2. Tabelul tickets

Câmp	Tip	Descriere
id	int / uuid (PK)	Identificator ticket
title	string	Titlu ticket
description	text	Conținut (posibil XSS dacă nu e tratat)
severity	enum	LOW / MED / HIGH
status	enum	OPEN / IN_PROGRESS / RESOLVED
owner_id	FK → users.id	Cine a creat ticket-ul
created_at	timestamp	Data creării
updated_at	timestamp	Ultima modificare

owner_id este **cheia pentru prevenirea IDOR**.

Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro

3. Tabelul audit_logs

Câmp	Tip	Descriere
id	int / uuid (PK)	Identificator eveniment
user_id	FK → users.id	Cine a făcut acțiunea
action	string	LOGIN / CREATE_TICKET / etc
resource	string	ticket / auth / audit
resource_id	string	ID-ul resursei afectate
timestamp	timestamp	Când s-a întâmplat
ip_address	string	IP (pentru realism)

Relațiile dintre tabele (foarte important pentru securitate)

Relație	De ce contează
users → tickets (1:N)	Permite verificarea ownership-ului
users → audit_logs (1:N)	Cine a făcut ce
tickets → audit_logs (1:N)	Ce s-a întâmplat cu un ticket

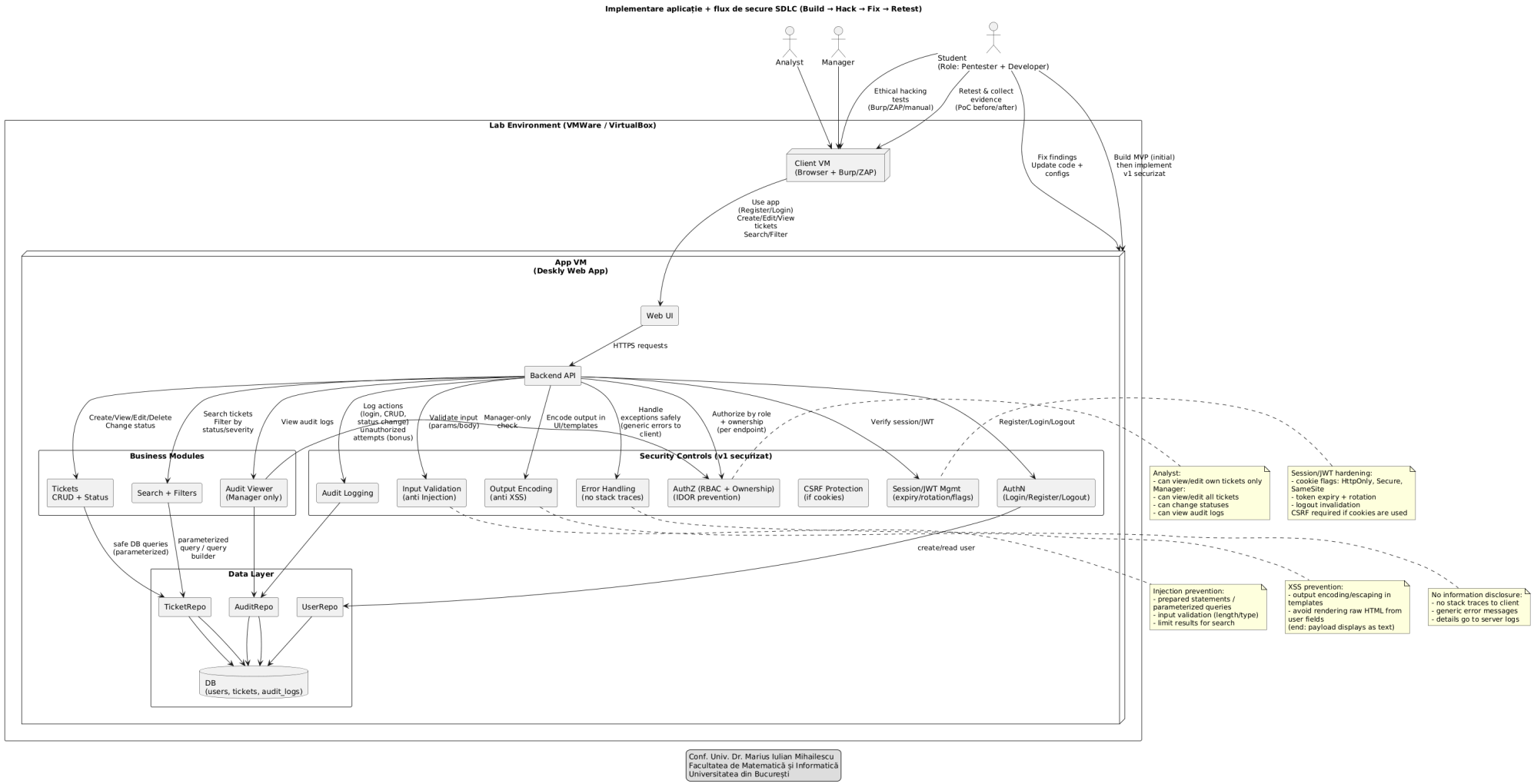
Cum este folosită DB în fluxul de securitate

Funcționalitate	Ce verifici în DB
Login	User + password_hash
View ticket	ticket.owner_id == user.id
Edit ticket	Rol + ownership
Search	Query parametrizat
Audit	Insert la fiecare acțiune
Brute force	Cont blocat după N încercări

Ce DB puteți utiliza?

Opțiune	De ce e bună
PostgreSQL	SQL real, bun pentru injection labs
MySQL	Foarte comun în industrie
SQLite	Simplu, local, suficient pentru proiect
MongoDB	Acceptabil dacă vrei NoSQL injection

Recomandare pentru curs: **PostgreSQL sau SQLite**, ca să rămână ușor de rulat local

Curs: Dezvoltarea Aplicațiilor Software Securizate
Facultatea de Matematică și Informatică
Universitatea din București

Conf. Univ. Dr. Marius Iulian Mihăilescu
marius-iulian.mihailescu@unibuc.ro